Automating Java Maven Build with Jenkins

This guide outlines how to replicate the setup and operations detailed in your assignment for building a Java Maven project and automating it with Jenkins.

---

1. Project Structure & Local Setup

To begin, you need to manually structure your Maven project. Maven follows a standard directory layout that is critical for the build tools to function.

Directory Layout

Create a folder named `maven-calculator-app` and set up the following hierarchy:

- `pom.xml` (Root directory)
- `src/main/java/com/example/App.java`
- `src/test/java/com/example/AppTest.java`

The Maven Configuration (`pom.xml`)

This file defines your project's dependencies, such as **JUnit** for testing.

XML

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>calculator</artifactId>
    <version>1.0-SNAPSHOT</version>

    <dependencies>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.2</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
</project>
```

---

2. Implementation: Java Code & Tests

Application Code (`App.java`)

Define a class with basic arithmetic operations.

Java

```java
package com.example;

public class App {
    public int add(int a, int b) { return a + b; }
    public int subtract(int a, int b) { return a - b; }
    public int multiply(int a, int b) { return a * b; }
```

```
    public int divide(int a, int b) { return a / b; }
}
```

Unit Testing (`AppTest.java`)

Create tests to verify that your calculator logic works as expected.

Java

```java
package com.example;

import org.junit.Test;
import static org.junit.Assert.*;

public class AppTest {
    App calculator = new App();

    @Test
    public void testAdd() {
        assertEquals(10, calculator.add(5, 5));
    }

    @Test
    public void testSubtract() {
        assertEquals(0, calculator.subtract(5, 5));
    }
}
```

---

3. Local Build Operations

Open your terminal in VS Code and run these Maven commands to verify the build:

1. **Compile:** `mvn compile` — Checks for syntax errors and generates bytecode.
2. **Test:** `mvn test` — Executes your JUnit tests.
3. **Build/Install:** `mvn clean install` — Cleans previous builds, runs tests, and packages the project into a JAR file.

---

4. Jenkins Automation (CI/CD)

Once your code is pushed to a Git repository, you can automate the process using **Jenkins**.

Setup a Pipeline

1. **Create Project:** In Jenkins, select **New Item** and choose **Pipeline**.
2. **Jenkinsfile:** Add a file named `Jenkinsfile` to your Git root to define the automation steps.

Example Jenkinsfile

Groovy

```groovy
pipeline {
    agent any
    stages {
        stage('Checkout') {
            steps {
                git 'https://github.com/adityagupta2004/maven-calculator-app.git'
            }
        }
        stage('Build') {
            steps {
                sh 'mvn clean install'
```

```
            }
        }
    }
}
```

Execution

Click **Build Now** in Jenkins. If successful, you will see a "Finished: SUCCESS" status in the **Console Output**.

How would you like to customize your Jenkins pipeline—for example, would you like to add automated deployment or notifications

In the context of DevOps and your assignment, a **Jenkinsfile** is a text file that contains the definition of a Jenkins Pipeline and is checked into source control.

Checking it into your **Git root** means placing the file in the top-level directory of your repository (the same folder as your `pom.xml`).

What "Automation Steps" Means

In your project, the Jenkinsfile tells Jenkins exactly what to do when it detects a change in your code. It breaks the workflow into **stages**:

- **Checkout:** Jenkins pulls the latest code from your GitHub repository.
- **Build/Compile:** It runs `mvn compile` to ensure the Java code is valid.
- **Test:** It runs `mvn test` to execute your JUnit tests in `AppTest.java`.
- **Install:** It runs `mvn clean install` to package your application into a JAR file.

Why use a Jenkinsfile?

Instead of manually clicking buttons in a web interface, the "Pipeline as Code" approach allows you to:

1. **Version Control:** Track changes to your build process just like you track changes to your Java code.
2. **Consistency:** Ensure the build runs the exact same way on the Jenkins server as it does on your local MacBook.
3. **Audit Trail:** See exactly who changed the build steps and when.

Example for your Project

Based on your successful build log, your Jenkinsfile effectively automates this sequence:

Groovy

```
pipeline {
    agent any // Run on any available Jenkins worker
    stages {
        stage('Build and Test') {
            steps {
                // This replaces you manually typing commands in the terminal
                sh 'mvn clean install'
            }
        }
    }
}
```

When you "run the pipeline," Jenkins reads this file, executes the commands, and provides the **Success** or **Failure** status you see in your report.

Since you've successfully run build #1, would you like to know how to trigger this build automatically every time you push code to GitHub?

To set this up as a complete beginner, think of this process as building a LEGO set. You need the pieces (Java code), the instructions (Maven), and a robot to build it for you automatically (Jenkins).

Here is the step-by-step breakdown.

Step 1: Set Up Your Project Folder

Before writing code, you must organize your computer so the tools know where to look.

1. **Create a Main Folder:** Name it `maven-calculator-app`.
2. **Create Subfolders:** Inside that folder, create this exact path: `src/main/java/com/example`.
3. **Create a Test Folder:** Also create: `src/test/java/com/example`.

   Why? Maven is "opinionated." It expects source code in `src/main` and tests in `src/test`. If you put them elsewhere, the build will fail.

Step 2: The "Brain" of the Project (`pom.xml`)

Create a file named `pom.xml` in your main folder. This is a configuration file that tells Maven:

- What your project is named (`calculator`).
- What extra tools it needs (like **JUnit** for testing).

Copy this into `pom.xml` :

XML

```xml
<project>
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>calculator</artifactId>
    <version>1.0-SNAPSHOT</version>

    <dependencies>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.2</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
</project>
```

Step 3: Write the Java Logic

Create `App.java` inside `src/main/java/com/example`. This is where you write the actual math for your calculator.

Java

```java
package com.example; // Tells Java which folder this is in

public class App {
    public int add(int a, int b) {
        return a + b;
    }
}
```

Step 4: Write the "Safety Check" (Unit Test)

Create `AppTest.java` inside `src/test/java/com/example`. This file checks if your math is correct.

Java

```
package com.example;
import org.junit.Test;
import static org.junit.Assert.*;

public class AppTest {
    @Test
    public void testAdd() {
        App calc = new App();
        assertEquals(10, calc.add(5, 5)); // We expect 10
    }
}
```

Step 5: Test it Locally

Open your terminal in VS Code and type:

`mvn clean install`.

- **Clean:** Deletes old, messy build files.
- **Install:** Compiles the code and runs the tests.
- **Success:** You should see **BUILD SUCCESS** in green letters.

Step 6: Automate with Jenkins

Now, instead of you typing commands, we want Jenkins to do it.

1. The Jenkinsfile

Create a file named `Jenkinsfile` (no file extension) in your main folder. This is the "script" for the robot.

Groovy

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                sh 'mvn clean install' // The robot types this for you
            }
        }
    }
}
```

2. Configure Jenkins

1. **Open Jenkins** in your browser (usually `localhost:8080`).
2. **New Item:** Give it a name like `calculator-final` and select **Pipeline**.
3. **Pipeline Definition:** Point it to your Git repository URL.
4. **Build Now:** Click this button.

Jenkins will now download your code, run the Maven command, and show you a green checkmark if everything passed.

Since you are running this on a MacBook Air, did you already install the "Jenkins" application and "Maven" using a tool like Homebrew?